Probabilistic Inference

CSC 5666: Advanced Machine Learning, Fall 2026
Professor Austin P. Wright: Course Notes

Abstract
Introduction to exact probabilistic inference with hidden Markov models, variable elimination, and belief propagation, followed by a brief look at loopy belief propagation. See Chapters 7–9 of Murphy's textbook.

Contents

I Inference Overview 1
II Message Passing 2
Belief Propagation on Chains 3
Variable Elimination 10
Belief Propagation on Factor Graphs 13

I Inference Overview

We have moved back and forth between models, distributions, and optimization. In a Bayesian model, the uncertain parts of the problem—parameters, predictions, or hidden states—are represented by random variables. The observed data are the values we condition on.

Inference means using Bayes' rule to compute a posterior distribution, or some quantity under that posterior.

$$\begin{aligned} p(\theta \mid \mathcal{D}) &= \frac{p(\theta)p(\mathcal{D} \mid \theta)}{p(\mathcal{D})} \\ &= \frac{p(\theta)p(\mathcal{D} \mid \theta)}{\int p(\theta)p(\mathcal{D} \mid \theta)d\theta} \end{aligned} \tag{1}$$

The numerator is usually easy to evaluate. The difficult part is the normalizing integral over every possible value of θ.

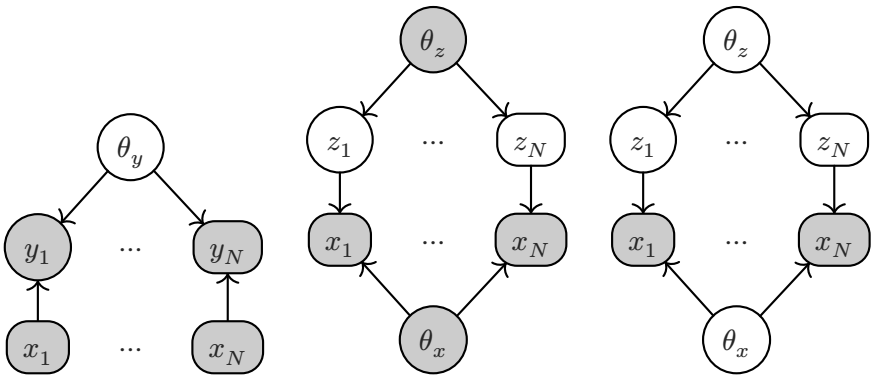
If we have the posterior (or as we will see at least a good estimate of it), we can use it to calculate expectations of functions of the parameters or unknown variables

$$\mathbb{E}[f(\theta) \mid \mathcal{D}] = \int f(\theta)p(\theta \mid \mathcal{D})d\theta \tag{2}$$

These functions will correspond to the actual quantities we care about, such as:

- Mean: $f(\theta) = \theta$
- Covariance: $f(\theta) = (\theta - \mathbb{E}[\theta])(\theta - \mathbb{E}[\theta])^\top$
- Predictions: $f(\theta) = p(y_{n+1} \mid \theta)$
- Expected Loss: $f(\theta) = \mathcal{L}(\theta, a)$

Optimization was built around differentiation; inference is mostly built around summation and integration. Graphical models let us reuse the same inference ideas across many models, as long as the graph and the local factors make those operations manageable. This is the usual idea behind *turning the Bayesian crank*.



(a) global (b) local (c) global + local

Figure 1: Three common inference patterns. (a) Global hidden variables, the Bayesian discriminative model $p(y_{1:N}, \theta_y \mid x_{1:N}) = p(\theta_y) \prod_n p(y_n \mid x_n; \theta_y)$. (b) Local hidden variables with known parameters, $p(x_{1:N}, z_{1:N} \mid \theta) = \prod_n p(z_n \mid \theta_z) p(x_n \mid z_n, \theta_x)$. (c) Local and global hidden variables, $p(x_{1:N}, z_{1:N}, \theta) = p(\theta_z) p(\theta_x) \prod_n p(z_n \mid \theta_z) p(x_n \mid z_n, \theta_x)$. Shaded nodes are observed. After textbook Figure 7.1.

The three pictures cover the common ways hidden variables show up in a model. In (a), one shared parameter θ_y is learned from all of the data. This is the usual Bayesian version of supervised learning: uncertainty about the model is represented by uncertainty about θ_y.

In (b), every case has its own hidden variable z_n. Once the parameters are fixed, the cases can be handled separately. Mixture models and the E-step of EM have this form. Panel (c) keeps both the shared parameters and the local hidden states. We will return to this pattern in variational inference.

II Message Passing

Textbook: Chapter 9, pp. 401–438

A graphical model tells us which calculations are local. On chains, trees, and other low-treewidth graphs, we can arrange those local calculations as a dynamic program and perform exact inference efficiently.

The intermediate result from one part of the graph is sent to the next part as a *message*. A message is a nonnegative function summarizing the evidence on one side of an edge; it need not be a probability distribution. Multiplying the incoming messages and normalizing gives a *belief*. This family of algorithms is therefore called *belief propagation*.

Belief Propagation on Chains

Textbook: §9.2, pp. 401–412

One of the simplest graph structures that is still extremely useful in many domains is a *chain* or *sequence*. Perhaps the most famous version of this structure is the Hidden Markov Model, which defines a sequence of hidden variables connected by a *transition function*, and for each hidden variable in the sequence there is a conditional *observation distribution* to an observed value. While this structure can be used in generality, we will focus on the illustrative case of discrete hidden and observed variables [1].

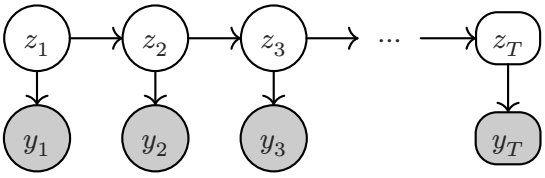


Figure 2: A hidden Markov model as a DPGM. z_t are the hidden states and y_t the observations. After textbook Figure 9.1.

[1] This allows us to represent the model with finite transition and observation matrices, which will help make more clear the discussion of performance and scale. Of course, HMM's are frequently used in both discrete and continuous contexts, and in fact have a wide range of extensions and generalizations for different domains.

A chain is a good place to start because the graph is simple but the main inference questions are already here: filtering, smoothing, and decoding. Later, variable elimination and belief propagation will recover the same calculations without relying on the special shape of an HMM.

Reading the joint from the graph gives one initial-state term, $T-1$ transition terms, and T observation terms. The graph makes two assumptions. The next state depends on the present state, not the whole past; and each observation depends only on its own hidden state.

Here π is the initial distribution and A is the transition matrix, with row i giving the next-state distribution from state i. The vector $\boldsymbol{\lambda}_t$ is local evidence: $\lambda_t(j)$ says how well state j explains the observation y_t. It is a likelihood, not a distribution over states, so it need not sum to one. Brute force would inspect K^T hidden paths. The recursions below avoid that enumeration.

$$p(y_{1:T}, z_{1:T}) = p(z_1)\left[\prod_{t=2}^{T} p(z_t \mid z_{t-1})\right] \times \left[\prod_{t=1}^{T} p(y_t \mid z_t)\right] \tag{3}$$

$$\pi_j = p(z_1 = j), \quad A_{ij} = p(z_t = j | z_{t-1} = i), \quad \lambda_t(j) = p(y_t | z_t = j) \tag{4}$$

Example: The Occasionally Dishonest Casino

Textbook: §9.2.1.1, pp. 402–403

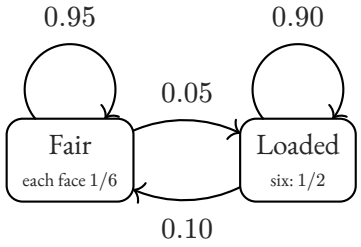


Figure 3: Casino HMM state transition diagram. After textbook Figure 9.2, adapted from Durbin et al. 1998, p. 54.

The casino usually uses a fair die, but sometimes swaps in a loaded die that rolls a six half the time. We see the rolls but not which die was used. The hidden state is therefore *fair* or *loaded*, and the observed value is the face of the die.

The large self-transition probabilities matter as much as the emission probabilities. Once the casino picks a die, it tends to keep it: the expected run lengths are $\frac{1}{1-0.95} = 20$ fair rolls and $\frac{1}{1-0.90} = 10$ loaded rolls. This lets several ordinary-looking rolls add up to evidence for a loaded stretch.

Try marking the loaded runs from the observation line before reading the hidden line. The first loaded run, rolls 11–14, has no six at all, while the two sixes at 15–16 came from the fair die. One roll is weak evidence; the sequence is the useful part.

$$
\begin{aligned}
A &= \begin{pmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{pmatrix} \\
p(y_t \mid z_t = \text{fair}) &= \text{Cat}(1/6, ..., 1/6) \\
p(y_t \mid z_t = \text{loaded}) &= \text{Cat}(1/10, ..., 1/10, 5/10)
\end{aligned} \tag{5}
$$

```
hid:  1111111111222211111111111111111111112222222221222211111111111111111111
obs:  13555345265533663163515515262321121134622212632642654223446453232423 61
```

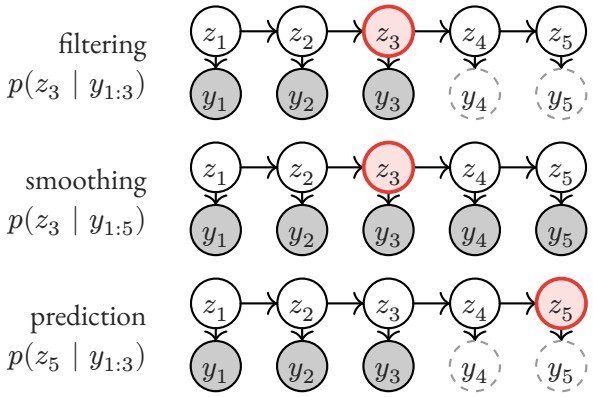


Figure 4: Which evidence each query uses, for $T = 5$. The queried state is outlined in red; shaded observations are conditioned on, dashed ones are not.

The three queries differ in which state we ask about and how much of the observation sequence we can use. Filtering uses data through time t, so it is the answer available to an online system. Smoothing looks back after the whole sequence has

arrived. Prediction starts from the filtered belief and pushes it forward without seeing new observations.

For smoothing, split the evidence at z_t. Given z_t, future observations do not depend on the observations already seen. Bayes' rule then gives a forward term, $p(z_t|y_{1:t})$, times a backward term, $p(y_{t+1:T}|z_t)$. The next two algorithms compute those two pieces.

$$\begin{aligned} \text{filtering:} \quad & p(z_t \mid y_{1:t}) \\ \text{smoothing:} \quad & p(z_t \mid y_{1:T}) \\ \text{prediction:} \quad & p(z_{t+h} \mid y_{1:t}), \quad h > 0 \end{aligned} \tag{6}$$

$$p(z_t = j \mid y_{1:T}) \propto p(z_t = j \mid y_{1:t})\, p(y_{t+1:T} \mid z_t = j) \tag{7}$$

The Forwards Algorithm

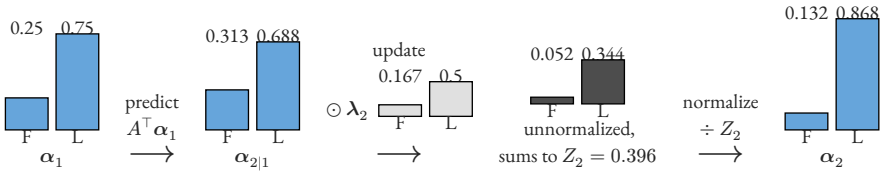


Figure 5: One step of the forwards algorithm on the casino, after two sixes in a row from a uniform start. Predict pushes yesterday's belief through A; update multiplies by the local evidence of a six, $\boldsymbol{\lambda} = [1/6, 1/2]$ (gray inset); normalizing divides by $Z_2 = p(y_2 \mid y_1)$.

The forwards algorithm repeats two small operations. First, *predict*: average yesterday's belief through the transition matrix. In the casino figure the loaded probability falls from 0.75 to about 0.69, since the loaded die might have been swapped out. Then *update*: multiply by the likelihood of today's roll and normalize. A second six raises the loaded probability to about 0.87. For the first observation, start with $\boldsymbol{\alpha}_1 \propto \pi \odot \boldsymbol{\lambda}_1$.

The normalizer Z_t is useful. It is the predictive probability of the new observation, $p(y_t|y_{1:t-1})$. By the chain rule, the product of these terms is the marginal likelihood, or equivalently $\log p(y_{1:T}) = \sum_t \log Z_t$. Normalizing at every step also avoids the rapid underflow of an unnormalized forward joint.

Each predict step is a matrix-vector product, so a dense K-state model costs $O(K^2T)$ rather than enumerating K^T paths. The filtered casino curve responds in the right places, but it is jumpy and tends to notice a loaded run late. It has not seen the future yet.

Discussion

Filtering is causal and smoothing is not. Name an application where you are only ever allowed the filtered distribution, and one where reporting it instead of the smoothed one would be a mistake. (Russell & Norvig, *AIMA* 4th ed., §14.2)

Textbook: §9.2.2, pp. 403–404

$$\begin{aligned}
\text{predict:}\quad & \alpha_{t|t-1}(j) \stackrel{\text{def}}{=} p(z_t = j \mid y_{1:t-1}) = \sum_i A_{ij}\, \alpha_{t-1}(i) \\
\text{update:}\quad & \alpha_t(j) \stackrel{\text{def}}{=} p(z_t = j \mid y_{1:t}) = \frac{1}{Z_t} \lambda_t(j)\, \alpha_{t|t-1}(j) \\
& Z_t \stackrel{\text{def}}{=} p(y_t \mid y_{1:t-1}) = \sum_j \lambda_t(j)\, \alpha_{t|t-1}(j) \qquad (8)
\end{aligned}$$

In matrix-vector form, with $\odot$ the elementwise product,

$$\boldsymbol{\alpha}_t = \text{normalize}(\boldsymbol{\lambda}_t \odot (A^\top \boldsymbol{\alpha}_{t-1})), \quad \log p(y_{1:T}) = \sum_{t=1}^{T} \log Z_t. \qquad (9)$$

Textbook §9.2.3.4, p. 407. For the unnormalized form, if observations were independent of the state we would have $p(z_t = j, y_{1:t}) = p(z_t = j) \prod_{i=1}^{t} p(y_i)$, which becomes exponentially small with t.

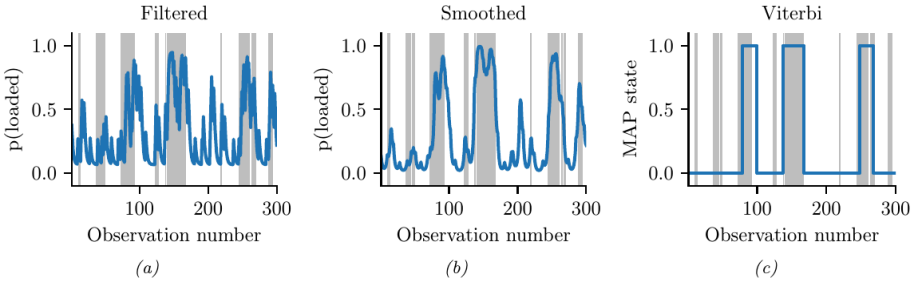


Figure 6: Inference in the dishonest casino over $T = 300$ rolls. Gray bars mark the times the loaded die was actually in use; blue lines show the posterior probability of the loaded state. (a) Filtered, $p(z_t \mid y_{1:t})$. (b) Smoothed, $p(z_t \mid y_{1:T})$. (c) The Viterbi (MAP) trajectory. Reproduced from Murphy, Textbook Figure 9.3, p. 404 (CC BY-NC-ND).

The Forwards-Backwards Algorithm

Textbook: §§9.2.3–9.2.5, pp. 404–409

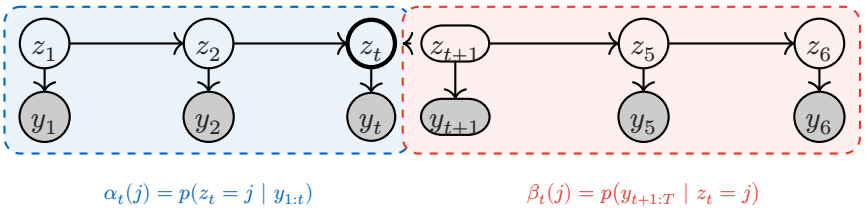


Figure 7: Smoothing splits the evidence at z_t. The forwards message α_t summarizes everything to the left, including y_t; the backwards message β_t summarizes everything to the right. Because z_t d-separates the two regions, the smoothed marginal is their normalized product.

The forward pass already summarizes everything to the left of z_t. For smoothing we also need a message from the right. Define $\beta_t(j)$ as the likelihood of all later observations if the current state is j. A scaled implementation stores $\beta_t^{\text{sc}} \propto \beta_t$ to avoid underflow.

To move one step left, try every possible next state, multiply its transition probability by its local evidence and its remaining backward likelihood, then sum. This gives $\boldsymbol{\beta}_{t-1} = A(\boldsymbol{\lambda}_t \odot \boldsymbol{\beta}_t)$, with $\boldsymbol{\beta}_T = \mathbf{1}$ because there is no future evidence after time T. Multiplying $\boldsymbol{\alpha}_t$ by either $\boldsymbol{\beta}_t$ or its scaled version and normalizing gives $\boldsymbol{\gamma}_t$.

The pairwise marginal $\xi_{t,t+1}$ records how much posterior mass uses each transition. Baum-Welch uses sums of γ as expected state counts and sums of ξ as expected transition counts. Both passes take $O(K^2T)$ time; storing the forward messages takes $O(KT)$ space. In the casino plot, the backward evidence makes the smoothed curve less jumpy and lines it up better with the hidden runs.

$$\begin{aligned} \beta_t(j) &\overset{\text{def}}{=} p(y_{t+1:T} \mid z_t = j) \\ \gamma_t(j) &\overset{\text{def}}{=} p(z_t = j \mid y_{1:T}) \propto \alpha_t(j)\,\beta_t(j), \\ \boldsymbol{\gamma}_t &= \text{normalize}(\boldsymbol{\alpha}_t \odot \boldsymbol{\beta}_t) \end{aligned} \tag{10}$$

$$\begin{aligned} \beta_{t-1}(i) &= p(y_{t:T} \mid z_{t-1} = i) \\ &= \sum_j p(y_{t+1:T} \mid z_t = j)\, p(y_t \mid z_t = j)\, p(z_t = j \mid z_{t-1} = i) \\ &= \sum_j \beta_t(j)\, \lambda_t(j)\, A_{ij} \end{aligned} \tag{11}$$

$$\boldsymbol{\beta}_{t-1} = A(\boldsymbol{\lambda}_t \odot \boldsymbol{\beta}_t), \quad \beta_T(i) = p(\varnothing \mid z_T = i) = 1 \tag{12}$$

Algorithm 1: Forwards-Backwards

$\triangleright$ Given π, A, and local evidence $\boldsymbol{\lambda}_{1:T}$
$\boldsymbol{\alpha}_1, Z_1 \leftarrow \text{normalize}(\boldsymbol{\lambda}_1 \odot \pi)$
for $t = 2, \ldots, T$ **do**
  $\boldsymbol{\alpha}_t, Z_t \leftarrow \text{normalize}(\boldsymbol{\lambda}_t \odot (A^\top \boldsymbol{\alpha}_{t-1}))$
end
$\boldsymbol{\beta}_T^{\text{sc}} \leftarrow \mathbf{1}$
for $t = T, \ldots, 2$ **do**
  $\boldsymbol{\beta}_{t-1}^{\text{sc}} \leftarrow \text{normalize}(A(\boldsymbol{\lambda}_t \odot \boldsymbol{\beta}_t^{\text{sc}}))$
end
for $t = 1, \ldots, T$ **do**
  $\boldsymbol{\gamma}_t \leftarrow \text{normalize}(\boldsymbol{\alpha}_t \odot \boldsymbol{\beta}_t^{\text{sc}})$
end
return $\boldsymbol{\gamma}_{1:T}, \log p(y_{1:T}) = \sum_t \log Z_t$

$$\begin{aligned} \xi_{t,t+1}(i,j) &\stackrel{\text{def}}{=} p(z_t = i, z_{t+1} = j \mid y_{1:T}) \\ &\propto \alpha_t(i)\, A_{ij}\, \lambda_{t+1}(j)\, \beta_{t+1}(j) \end{aligned} \tag{13}$$

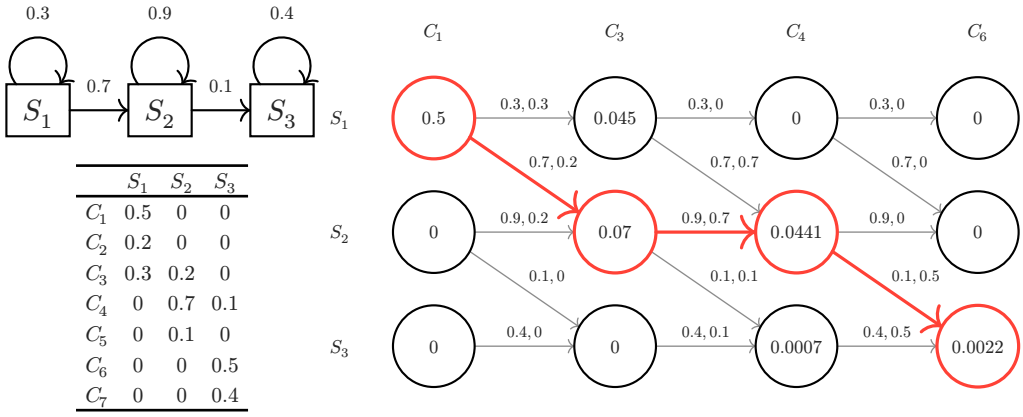
$$\underbrace{\sum_{z_{1:T}} p(y_{1:T}, z_{1:T})}_{O(K^T)\text{ terms}} \quad \longrightarrow \quad \underbrace{T \text{ steps of } A^\top \boldsymbol{\alpha}}_{O(K^2 T)} \tag{14}$$

Discussion

In the casino plot, the smoothed probability sometimes rises before the filtered probability. How can a belief about time t respond to later rolls, and which message carries that information?

Viterbi Algorithm

Textbook: §9.2.6, pp. 409–412



	S_1	S_2	S_3
C_1	0.5	0	0
C_2	0.2	0	0
C_3	0.3	0.2	0
C_4	0	0.7	0.1
C_5	0	0.1	0
C_6	0	0	0.5
C_7	0	0	0.4

Figure 8: Viterbi decoding in a three-state HMM for a single phone. Left: the state transition diagram and the emission probabilities of each symbol $C_1, \ldots, C_7$. Right: the trellis for observations (C_1, C_3, C_4, C_6). Each node shows $\delta_t(j)$ and each edge is labeled (transition probability, emission probability). The red path is recovered by traceback. After textbook Figure 9.6, adapted from Russell and Norvig.

Viterbi asks a different question. Instead of a marginal distribution at each time, it asks for the single most probable hidden path. Since the observed sequence is fixed, maximizing the posterior path probability is the same as maximizing its joint probability with the observations.

The recursion is the unnormalized forward recursion with the sum replaced by a max. $\delta_t(j)$ stores the score of the best prefix ending in state j. In the trellis, for example,

$$\delta_3(2) = \max(0.045 \cdot 0.7, 0.07 \cdot 0.9) \cdot 0.7 = 0.0441. \tag{15}$$

The winning incoming state is stored as a backpointer. After reaching the end, follow those pointers backward to recover one consistent path. The cost is again $O(K^2T)$ time and $O(KT)$ stored pointers; implementations normally use max-sum in log space.

A Viterbi path is a hard, whole-sequence decision. It need not minimize the number of state-by-state errors. On the textbook casino run, the per-time smoothed choices make fewer errors than the MAP path, even though the latter is the most probable complete sequence. Those decoders are minimizing different losses.

$$\begin{aligned} z^*_{1:T} &= \operatorname*{argmax}_{z_{1:T}} p(z_{1:T} \mid y_{1:T}) \\ &= \operatorname*{argmax}_{z_{1:T}} \Bigg[\log \pi(z_1) + \log \lambda_1(z_1) \\ &\qquad + \sum_{t=2}^{T} \Big(\log A_{z_{t-1} z_t} + \log \lambda_t(z_t) \Big) \Bigg] \end{aligned} \tag{16}$$

$$\delta_t(j) = \lambda_t(j) \max_i \big[\delta_{t-1}(i)\, A_{ij}\big], \quad \delta_1(j) = \pi_j \lambda_1(j)$$
$$a_t(j) = \operatorname*{argmax}_i \big[\delta_{t-1}(i)\, A_{ij}\big]$$
$$z_T^* = \operatorname*{argmax}_i \delta_T(i), \quad z_t^* = a_{t+1}(z_{t+1}^*) \tag{17}$$

Discussion

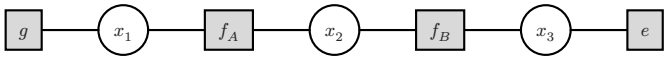
Why can the most probable complete sequence make more state-by-state errors than choosing the largest smoothed marginal at each time? What loss does each rule minimize?

Discussion

What changes in the forward recursion when the sum is replaced by a max? What extra information must we store to recover a path?

Variable Elimination

Textbook: §9.5, pp. 428–433



$g(x_1) = [0.6, 0.4]$
$e(x_3) = [0, 1]$

f_A	$x_2 = 0$	$x_2 = 1$
$x_1 = 0$	3	1
$x_1 = 1$	1	3

f_B	$x_3 = 0$	$x_3 = 1$
$x_2 = 0$	2	1
$x_2 = 1$	1	2

Figure 9: The running example for the rest of the section: a binary chain $x_1 - x_2 - x_3$ as a factor graph, with a prior factor g on x_1, pairwise factors f_A, f_B that prefer agreement, and an indicator factor e entering the evidence $x_3 = 1$.

The HMM recursions were tailored to a chain. Variable elimination uses the same distributive-law trick on any factor graph. In this small example, $e(x_3) = [0, 1]$ enters the observation $x_3 = 1$ as one more factor.

Brute force sums the product of all four factors over x_1 and x_3. We can instead push each sum next to the factors that mention its variable. The left branch becomes

$$\tau_1(x_2) = [2.2, 1.8] \tag{18}$$

and the right branch becomes

$$\tau_3(x_2) = [1, 2]. \tag{19}$$

Their product is $[2.2, 3.6]$, which normalizes to $[0.379, 0.621]$. The value 5.8 is the clamped partition value for these unnormalized potentials.

Each τ is a summary of a piece of the graph after its internal variable has been removed. That is also a message. If we eliminate the hidden states of an HMM from left to right, these summaries are the unnormalized forward messages.

$$\begin{aligned}\tilde{p}(x_2, e) &= \sum_{x_1}\sum_{x_3} g(x_1) f_A(x_1, x_2) f_B(x_2, x_3) e(x_3) \\ &= \underbrace{\left[\sum_{x_1} g(x_1) f_A(x_1, x_2)\right]}_{\tau_1(x_2)=[2.2,1.8]} \underbrace{\left[\sum_{x_3} f_B(x_2, x_3) e(x_3)\right]}_{\tau_3(x_2)=[1,2]} = [2.2, 3.6]\end{aligned}$$

$$p(x_2 \mid x_3 = 1) = [2.2, 3.6]/5.8 = [0.379, 0.621] \tag{20}$$

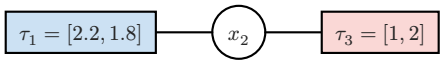
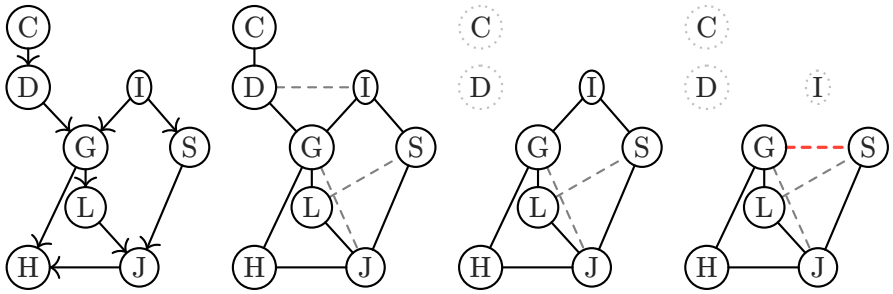


Figure 10: The running example after variable elimination has summed out x_1 and x_3: each eliminated branch has become a single factor on x_2.



(a) DPGM (b) moral (c) $-C, D$ (d) $-I$

Figure 11: Symbolic variable elimination on the student network (Coherence, Difficulty, Intelligence, Grade, SAT, Letter, Job, Happy). (a) The DPGM. (b) Its moralization; gray dashed edges join co-parents. (c) Eliminating C then D adds no edges. (d) Eliminating I joins its remaining neighbors G and S with a fill-in edge (red). After textbook Figure 9.18, adapted from Koller and Friedman Figure 9.10.

On a larger graph we do the same thing repeatedly. Choose a variable, multiply all current factors that mention it, sum it out, and put the resulting factor back into the pool. The table follows the order C, D, I, H, G, S, L for the query $p(J)$.

The graph strip shows the same work without the numbers. Eliminating C and then D creates no new adjacency among the remaining variables. Eliminating I does: its remaining neighbors G and S must be joined, producing both the red fill-in edge and the new factor $\tau_3(G, S)$. The graph therefore tells us the scope of the tables before we calculate them.

For a conditional query, first enter the observations as evidence factors, eliminate every variable that is neither observed nor queried, and normalize the final factor.

$$
\begin{aligned}
p(J) = & \sum_{L,S,G,H,I,D,C} \psi_C(C)\psi_D(D,C)\psi_I(I)\psi_G(G,I,D) \\
& \quad \times \psi_S(S,I)\psi_L(L,G)\psi_J(J,L,S)\psi_H(H,G,J) \\
= & \sum_{L,S} \psi_J(J,L,S) \sum_{G} \psi_L(L,G) \sum_{H} \psi_H(H,G,J) \\
& \quad \times \sum_{I} \psi_S(S,I)\psi_I(I) \sum_{D} \psi_G(G,I,D) \\
& \quad \times \sum_{C} \psi_C(C)\psi_D(D,C) \qquad (21)
\end{aligned}
$$

Step	Eliminate	Multiply	New factor
1	C	$\psi_C(C), \psi_D(D,C)$	$\tau_1(D)$
2	D	$\psi_G(G,I,D), \tau_1(D)$	$\tau_2(G,I)$
3	I	$\psi_S(S,I), \psi_I(I), \tau_2(G,I)$	$\tau_3(G,S)$
4	H	$\psi_H(H,G,J)$	$\tau_4(G,J)$
5	G	$\psi_L(L,G), \tau_4(G,J), \tau_3(G,S)$	$\tau_5(J,L,S)$
6	S	$\psi_J(J,L,S), \tau_5(J,L,S)$	$\tau_6(J,L)$
7	L	$\tau_6(J,L)$	$\tau_7(J) = p(J)$

Table 1: Eliminating the student network in the order C, D, I, H, G, S, L to compute $p(J)$. After textbook Table 9.1.

Discussion

In the elimination below, step 4 sums out H from $\psi_H(H,G,J)$ alone. If we only want $p(J)$, what is $\sum_H \psi_H(H,G,J)$ for a directed model, and which variables could we have pruned before starting? (Berkeley CS188, Bayes net inference lecture)

Complexity and Hardness

Textbook: §§9.5.2–9.5.4, pp. 430–433

The order matters because a variable's current neighbors become the scope of the temporary computation used to eliminate it. With the order in the table, eliminating G works over G, J, L, S. If G is eliminated first, its five neighbors are joined and the temporary table involves six binary variables. Both orders return the same answer; one creates much larger tables.

For finite tabular factors with at most K values per variable, an order of induced width w costs roughly $O(NK^{w+1})$. Treewidth is the smallest induced width over all orders. Chains and trees have treewidth one. Finding the best order is itself hard, so implementations usually use a heuristic such as min-fill.

Exact marginal inference is hard in general; low treewidth is the important easy case for discrete graphical models. Domain size and the algebra of the factors matter too, especially for continuous models. There is also repeated work if we run ordinary variable elimination again for every query. Belief propagation keeps the intermediate results and reuses them.

Discussion

Chains and trees have treewidth 1. What is the treewidth of an $n \times n$ grid, and what does that imply about exact inference in an image-sized Ising model? (Stanford CS228 notes, "Variable elimination"; Koller & Friedman §9.4)

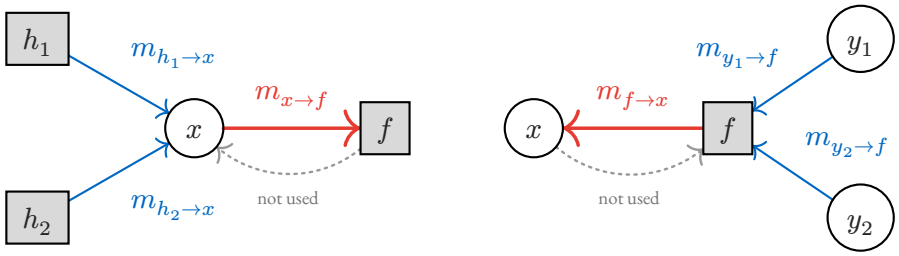
$$\text{cost}(\prec) = \sum_{c \in \mathcal{C}(G_\prec)} K^{|c|} = O(NK^{w_\prec + 1})$$
$$w_\prec = \max_{c \in \mathcal{C}(G_\prec)} |c| - 1, \quad \text{treewidth} = \min_{\prec} w_\prec \tag{22}$$

Discussion

From the moral graph, choose a cheap variable to eliminate next and an expensive one. What table would each elimination create?

Belief Propagation on Factor Graphs

Textbook: §§9.3–9.4, 9.6, pp. 412–428, 434



(a) variable → factor: **multiply** what every *other* factor said

(b) factor → variable: multiply by f, **sum out** every *other* variable

Figure 12: The two sum-product rules. Both rules exclude the recipient's own message (dashed): a node never echoes information back to where it came from. After Kschischang, Frey, and Loeliger 2001, Figure 6 (textbook Figure 9.10).

Sum-product can be read as variable elimination cached on the edges. A variable sends a factor the product of the messages from its other neighboring factors. A factor multiplies its table by the messages from its other variables, sums those variables out, and sends the remaining function to the recipient.

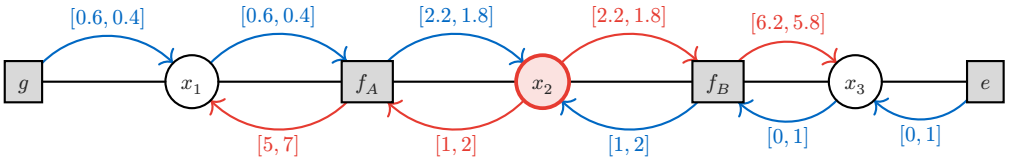
The phrase "other neighbors" is important. A node does not immediately send a message back along the edge it came from; doing so would count the same evidence twice. Messages are just nonnegative functions. They do not have to be normalized probability distributions.

On a tree, after messages have crossed every edge in both directions, multiplying the incoming messages at a variable gives its exact marginal up to normalization. The analogous factor belief gives the exact marginal over that factor's scope. Leaf variables begin with an all-ones message, while a leaf factor can send its own table. With bounded factor size, the total work is linear in the number of edges.

$$p(\boldsymbol{x}) = \frac{1}{Z} \prod_{f \in \mathcal{F}} f(\boldsymbol{x}_f) \tag{23}$$

$$
\begin{aligned}
m_{x\to f}(x) &= \prod_{h\in\,\mathrm{nbr}(x)\setminus\{f\}} m_{h\to x}(x) \\
m_{f\to x}(x) &= \sum_{\boldsymbol{x}_f\setminus x} f(\boldsymbol{x}_f) \prod_{x'\in\,\mathrm{nbr}(f)\setminus\{x\}} m_{x'\to f}(x') \\
\mathrm{bel}_x(x) &\propto \prod_{f\in\,\mathrm{nbr}(x)} m_{f\to x}(x) \\
\mathrm{bel}_f(\boldsymbol{x}_f) &\propto f(\boldsymbol{x}_f) \prod_{x\in\,\mathrm{nbr}(f)} m_{x\to f}(x)
\end{aligned}
\tag{24}
$$

Worked Example: Sum-Product on a Chain



$b_1 = [0.517, 0.483]$ $b_2 = [0.379, 0.621]$ $b_3 = [0, 1]$

Figure 13: Sum-product on the running example, rooted at x_2. Blue messages collect toward x_2; red messages distribute back out. Each reply excludes the message received on that edge.

Pick x_2 as the root and collect messages from both ends. From the left, g sends $[0.6, 0.4]$ through x_1. The factor f_A then sums out x_1 and sends $[2.2, 1.8]$. From the right, the evidence sends $[0, 1]$ through x_3, and f_B sends $[1, 2]$. Their product at x_2 is $[2.2, 3.6]$, giving the belief $[0.379, 0.621]$ after normalization.

Now distribute back out. Toward x_1, x_2 sends only the information that arrived from the right, namely $[1, 2]$. After f_A sums over x_2, its message is $[5, 7]$. Multiplying by g gives $[3.0, 2.8]$, or $[0.517, 0.483]$. The same calculation toward x_3 gives $[0, 1]$, as it must because x_3 was observed.

These are the same intermediate factors produced by variable elimination, but now they stay on the edges. The four-row table below gives a direct check: its entries sum to 5.8, and summing the appropriate rows reproduces the variable beliefs.

(x_1, x_2)	product	$\tilde{p}$	$p(x_1, x_2 \mid x_3 = 1)$
$(0, 0)$	$0.6 \cdot 3 \cdot 1$	1.8	0.310
$(0, 1)$	$0.6 \cdot 1 \cdot 2$	1.2	0.207
$(1, 0)$	$0.4 \cdot 1 \cdot 1$	0.4	0.069
$(1, 1)$	$0.4 \cdot 3 \cdot 2$	2.4	0.414
		$Z = 5.8$	

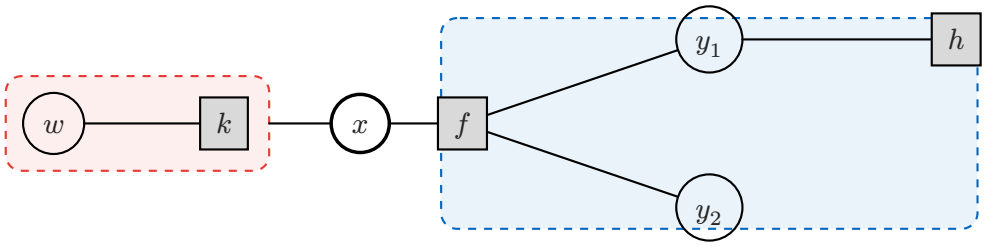
Table 2: Brute-force check. With $x_3 = 1$ fixed, the unnormalized joint $g(x_1) f_A(x_1, x_2) f_B(x_2, 1)$ has four terms. It is also the factor belief $\mathrm{bel}_{f_A}(x_1, x_2)$ before normalization.

Discussion

Remove the evidence factor e. What message does f_B send to x_2? Why does an unobserved leaf carry no information?

Exact Inference on Trees

Textbook: §§9.3.2, 9.5.5–9.6, pp. 414–415, 433–434



$m_{k\to x}(x) = \sum_w k(x, w)$ $m_{f\to x}(x) = \sum_{y_1, y_2} f(x, y_1, y_2) h(y_1)$

Figure 14: On a tree, the message a factor sends toward x is the sum over the whole subtree behind it (blue region), and different neighbors' subtrees (blue, red) share no variables. Multiplying all messages into x therefore sums the full joint over everything except x.

For a tree, choose any node as a root. In the collect pass, messages move from the leaves toward the root. In the distribute pass, they move back out. Each edge carries one message in each direction, after which every variable and factor belief is exact.

The reason is visible in the shaded-subtree figure. A message across an edge is the sum of the product of all factors on one side of that edge. Different branches of a tree share no variables, so the distributive law lets us compute each branch separately and multiply the summaries at the receiving node. The root changes the schedule, not the answer.

The HMM is the same algorithm on a chain-shaped factor graph. The message arriving from the left transition is the forward prediction, the message from the right is β_t, and their product with the local evidence gives γ_t. In a linear-Gaussian chain, the same pattern gives the Kalman smoother.

For graphs with cycles, exact inference can still be organized by a junction tree: combine variables into cliques, connect those cliques as a tree, and pass messages between them. This reuses work across marginals, but its clique tables remain exponential in treewidth.

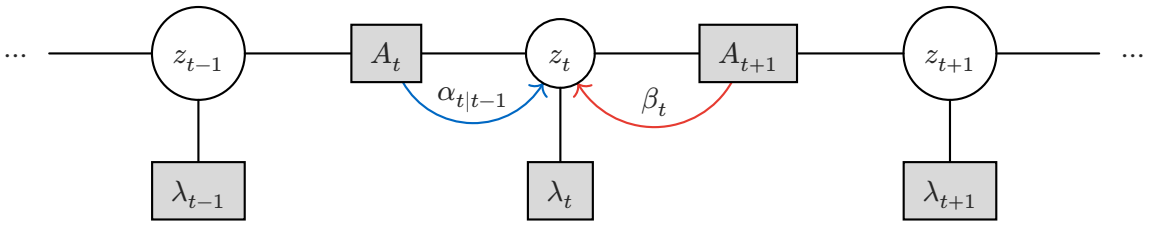


Figure 15: The HMM as a factor graph. Square nodes are the transition factors $A_t(z_{t-1}, z_t) = A_{z_{t-1}z_t}$ and the local evidence factors λ_t. Sum-product messages into z_t from the left and right transition factors are exactly the forwards prediction $\alpha_{t|t-1}$ and the backwards message β_t.

$$
\begin{aligned}
m_{z_{t-1}\to A_t}(z_{t-1}) &= \lambda_{t-1}(z_{t-1})\, m_{A_{t-1}\to z_{t-1}}(z_{t-1}) \propto \alpha_{t-1}(z_{t-1}) \\
m_{A_t\to z_t}(z_t) &= \sum_{z_{t-1}} A_{z_{t-1}z_t}\, m_{z_{t-1}\to A_t}(z_{t-1}) \propto \alpha_{t|t-1}(z_t) \\
m_{A_{t+1}\to z_t}(z_t) &= \sum_{z_{t+1}} A_{z_t z_{t+1}}\, \lambda_{t+1}(z_{t+1})\, m_{A_{t+2}\to z_{t+1}}(z_{t+1}) = \beta_t(z_t) \\
\mathrm{bel}_{z_t}(z_t) &\propto \alpha_{t|t-1}(z_t)\, \lambda_t(z_t)\, \beta_t(z_t) \propto \gamma_t(z_t) \qquad (25)
\end{aligned}
$$

Max-Product

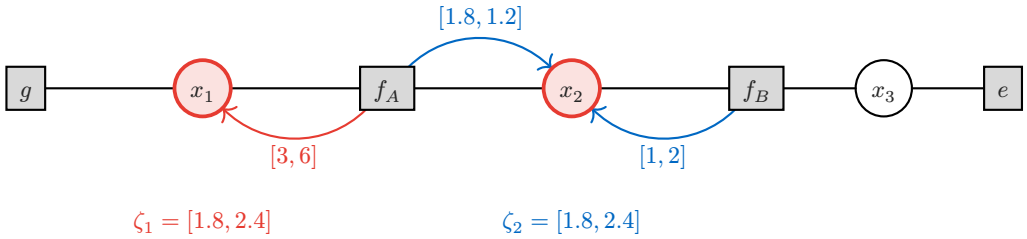


Figure 16: Max-product on the running example. Each max-marginal peaks at 2.4 and points to the MAP assignment $(x_1, x_2) = (1, 1)$.

Max-product changes one operation: factor nodes maximize over their other variables instead of summing them out. The resulting max-marginal $\zeta_i(k)$ is the score of the best complete assignment having $z_i = k$. On an HMM this is Viterbi.

For the running example, f_A sends $[1.8, 1.2]$ and f_B sends $[1, 2]$, so $\zeta_2 = [1.8, 2.4]$. Passing back toward x_1 gives $[3, 6]$, and multiplication by g again produces $[1.8, 2.4]$. Both max-marginals point to the joint MAP assignment $(1, 1)$, whose unnormalized score is 2.4.

Sum-product answers a different question. Its marginal modes are $(0, 1)$: $x_1 = 0$ is slightly more likely after summing over x_2, even though the best joint assignment uses $x_1 = 1$. Marginal modes minimize expected per-variable mistakes; joint MAP minimizes the chance that the whole assignment is wrong. If max-marginals tie, independent choices may not form a valid MAP assignment, so keep backpointers.

Discussion

The sum-product answer does not depend on which node we choose as the root. Why not? And which step of the argument would fail if the graph had a single cycle? (MacKay, *Information Theory, Inference, and Learning Algorithms*, Ch. 16, "counting soldiers")

Textbook: §9.3.3, pp. 415–417

$$m_{f\to x}(x) = \max_{\boldsymbol{x}_f \setminus x} f(\boldsymbol{x}_f) \prod_{x' \in \,\mathrm{nbr}(f)\setminus\{x\}} m_{x'\to f}(x') \tag{26}$$

$$\gamma_i(k) = \sum_{z_{-i}} p(z_i = k, z_{-i} \mid y),\ \hat{z}_i = \operatorname*{argmax}_k \gamma_i(k) \quad \text{(MPM)}$$

$$\zeta_i(k) = \max_{z_{-i}} \tilde{p}(z_i = k, z_{-i}, y),\ \tilde{z}_i = \operatorname*{argmax}_k \zeta_i(k) \quad \text{(MMM)}$$

$$z^* = \operatorname*{argmax}_z p(z \mid y) \quad \text{(MAP)} \tag{27}$$

Loopy Belief Propagation

Textbook: §9.4, pp. 417–428

J. S. Yedidia, W. T. Freeman, and Y. Weiss, "Constructing free-energy approximations and generalized belief propagation algorithms", *IEEE Trans. Information Theory* 51(7), 2005.

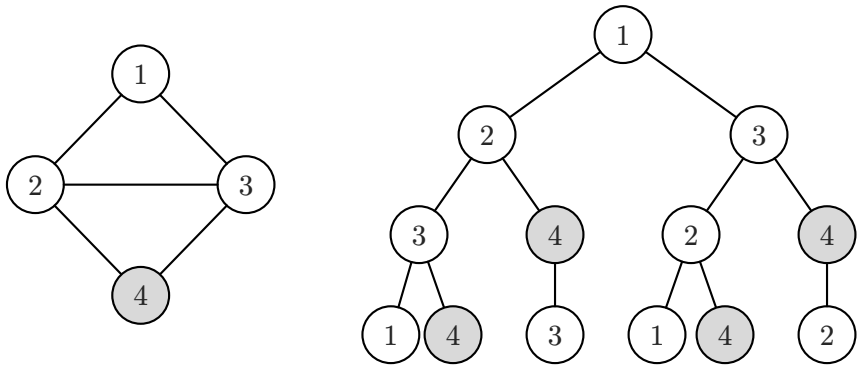


Figure 17: (a) A simple loopy graph. (b) Its computation tree rooted at node 1 after three rounds of message passing. Evidence at node 4 (shaded) reaches node 1 along two different branches, and the original variables recur in several places. After textbook Figure 9.12, from Wainwright and Jordan 2008.

The local message rules still make sense on a graph with cycles. Initialize the messages, update them repeatedly, and stop if they settle down. This is *loopy belief propagation*. Unlike the tree case, it may oscillate, and a fixed point need not give the exact marginals.

The computation tree explains the trouble. After a few rounds, loopy BP at node 1 behaves like exact BP on the unrolled tree in the figure. The same original variable appears in several places, so its evidence can return along more than one route. Thinking of this as "echoing" or double counting is useful: weak, distant influence may die away, while strong influence can be repeated and make beliefs unstable or too sharp. Sparse coding graphs are often locally tree-like, which is one reason BP works remarkably well for decoding.

Damping mixes a new message with its old value. Asynchronous and residual schedules change which message is updated next. These tricks often help convergence, but they do not make the answer exact. The Ising example shows both points: one schedule oscillates, while another converges to the wrong value for X_{17}.

There is a useful bridge to the next lecture. Fixed points of loopy BP correspond to stationary points of the Bethe free energy, an approximation to the exact variational objective. This does not mean the ordinary message updates steadily minimize that objective, but it does put loopy BP in the same larger family of approximate inference ideas.

$$m_{f\to x}^{(k)} = \lambda\, \tilde{m}_{f\to x}^{(k)} + (1-\lambda)\, m_{f\to x}^{(k-1)}, \quad 0 < \lambda \le 1 \tag{28}$$

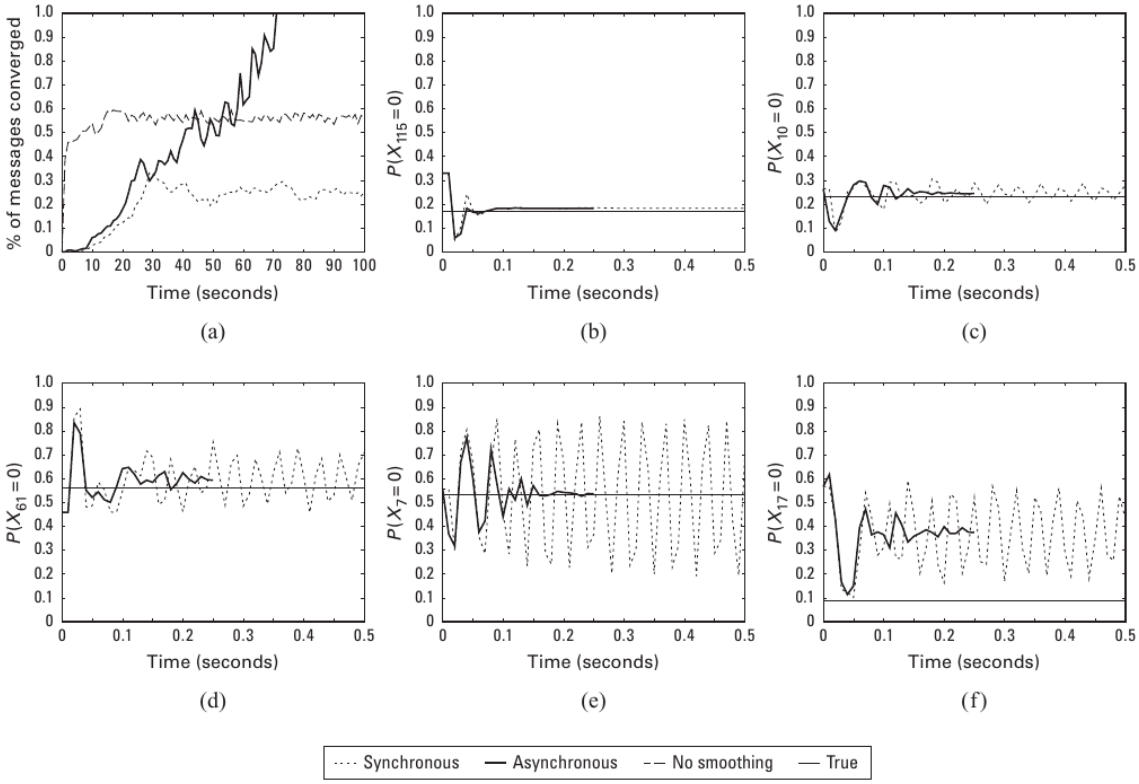


Figure 18: Loopy BP on an 11×11 Ising grid with random couplings $w_{ij} \sim \mathrm{Unif}(-C, C)$, $C = 11$. (a) Fraction of messages converged over time for damped synchronous (dotted), undamped asynchronous (dashed), and damped asynchronous (solid) schedules. (b–f) Marginal beliefs of individual nodes over time against the true marginal (straight line). Reproduced from Murphy, Textbook Figure 9.13, p. 422, originally Koller and Friedman Figure 11.C.1 (CC BY-NC-ND).

Discussion

In panel (f), the messages converge but the belief for X_{17} is wrong. What does this say about using convergence as evidence that inference succeeded?

Discussion

How should a smaller damping value λ change the speed and stability of the updates?

Discussion

Exact inference is fast on a chain but hard on a general graph. What changes as treewidth grows? Besides graph width, what can make a local elimination step expensive?

Algorithm	**Graph**	**Computes**	**Cost**	**Exact?**
Forwards	chain	filtered α_t, evidence $p(y_{1:T})$	$O(K^2T)$	yes
Forwards-backwards	chain	smoothed γ_t, pairs $\xi_{t,t+1}$	$O(K^2T)$	yes
Viterbi	chain	MAP sequence	$O(K^2T)$	yes
Variable elimination	any	one marginal (or Z)	$O(NK^{w+1})$	yes
Sum-product BP	tree	all marginals, Z	$O(\lvert\mathcal{E}\rvert\ K^{d_{\max}})$	yes
Max-product BP	tree	max-marginals, MAP	$O(\lvert\mathcal{E}\rvert\ K^{d_{\max}})$	yes
Junction tree	any	all clique marginals	$O(NK^{w+1})$	yes
Loopy BP	any	approximate marginals	$O(\lvert\mathcal{E}\rvert\ K^{d_{\max}})$ per sweep	no

Table 3: The algorithms of this section at a glance. K is the number of states per variable, T the chain length, N the number of variables, w the induced width of the elimination order, and $d_{\max}$ the largest factor arity.

All of these methods avoid repeating the same sums. On chains and trees this gives efficient exact inference. On wide graphs the intermediate factors become too large, and some continuous factors are hard to integrate even when the graph is simple.

Next we give up exactness. Variational inference fits a tractable distribution q by optimization; Monte Carlo uses samples. The next lecture starts by asking what the best fully factorized q looks like.